

Herramientas integradas dentro de los agentes

Claude Certified Developer, Foundation (CCDV-F) · Masterclass UST

ES

Sesión 12 de 12 · Notas en español (es-ES)

La sesión 12 (27 de agosto de 2026) cerró el **uso de herramientas integradas** en el Claude Agent SDK. No escribís Read/Write/Edit/Bash vosotros. Las *permitís* en el agente, dais un objetivo y dejáis que Claude Code ejecute mientras Claude decide. El material estaba en el GitHub del grupo; el PDF del día anterior aún se subía al empezar la llamada.

Por qué existen las integradas

Las herramientas a medida del módulo 10 eran relojes y envoltorios HTTP. Los agentes que trabajan un repositorio necesitan primitivas de sistema de ficheros y de shell que el SDK ya trae. Habilitar una herramienta es una decisión de permiso tanto como un flag de función: el capítulo 02 de este módulo lo trata como seguridad. Este capítulo es capacidad.

LOS NOMBRES SE MUEVEN

Los identificadores exactos (Bash frente a Shell, Monitor frente a un ayudante de streaming) hay que comprobarlos en la documentación vigente de **Claude Code** y del Agent SDK. La pedagogía sigue el cuaderno del 27 de agosto.

Bash frente a Monitor

Permitir **Bash** deja al agente ejecutar scripts, comandos del SO y Git. El prompt de demostración pedía un informe de sistema: versión de Python (la máquina del formador mostraba la familia 3.13), directorio de trabajo, disco, fecha, presentado de forma legible. Claude eligió los comandos; Bash los corrió.

Monitor se enseñó como compañero para trabajo largo o en streaming: un script en segundo plano, cada línea de stdout alimentada a Claude según aparece, en lugar de un bloque al terminar. Uno o dos comandos cortos: basta Bash. Una serie de pasos donde un error de permiso debe verse *a mitad*: habilitad Monitor con Bash. La demo de build_script.py imprimía una línea cada tres segundos; se pidió al agente reaccionar en una frase por línea antes de la siguiente.

¿Quién escribe el bucle de reintento?

Mritunjay quería ver un bucle explícito «si hay error, edita» en Python. La respuesta de Vijay: **el modelo posee el bucle** dentro del iterador de mensajes de query. Habilitáis Monitor, Bash, Read y Edit; escribís un prompt que dice «si falla, lee el fichero, explica el error, edita, vuelve a monitorizar». La demo flaky_build.py dividía por cero a propósito; tras confusión de rutas (relativa frente a absoluta, mal directorio de trabajo) quedó la lección: el prompt más las herramientas permitidas, no una máquina de estados a mano, impulsaban el recuperar-y-repetir.

Glob frente a Grep

Son las herramientas de búsqueda que usa Claude Code cuando inspecciona un proyecto antes de editar.

- **Glob** — encontrar ficheros por nombre/extensión en un árbol («todos los `.py` bajo `search_demo`, rutas completas, contarlos»).
- **Grep** — encontrar líneas dentro de ficheros (comentarios `TODO`, cabeceras `^def`). Se espera regex.

Un objetivo combinado: Glob de todos los Python, Grep de sus definiciones de función, informe limpio. Esa instantánea es contexto para Write/Edit posteriores: la misma razón por la que no debéis volcar un monorepositorio entero en un prompt (lógica RAG, edición sistema de ficheros).

Búsqueda web frente a obtención web

Herramienta	Analogía del formador	Qué vuelve a Claude
Web search	Consulta tipo Google	Una caza sintetizada sobre resultados web en vivo (no el corte de entrenamiento del modelo)
Web fetch	Rastrear una URL	La página (o fichero) de una dirección conocida, descargada para extraer

Sin búsqueda, Claude responde desde parámetros que pueden llevar meses. Con búsqueda, la herramienta del *cliente* actúa como navegador y el *servidor* (LLM) formatea. El trueque: los resultados se vuelven **tokens de entrada**. Sed específicos o compráis una descarga grande. Fetch es más barato cuando ya tenéis la URL. La demo buscó ejemplos Python del Claude Agent SDK y apuntó notas hacia Markdown.

Ask user question (humano en el bucle)

Cuando el objetivo necesita una elección humana (formato de salida, multi-selección o no), habilitad **AskUserQuestion** y una retrollamada `can_use_tool`. El cuaderno guardaba preguntas en JSON (opciones, si es multi-selección). El agente pausa, vuestra retrollamada recoge la respuesta, el bucle sigue. Es la misma familia que los avisos de permiso, pero para preguntas de producto en lugar de «¿puedo ejecutar `rm`?».

Ángulo de examen

Si el escenario es «documentación última del Agent SDK», elegid búsqueda web (u obtención de una URL de docs conocida). Si es «listad cada `TODO` en `src/`», elegid Grep (y quizá Glob antes). Si es «ejecutad una build larga y comentad según aparecen los registros», elegid Monitor+Bash. Si es «versión actual de Python en este portátil», elegid Bash, no una herramienta reloj a medida.

Documentación oficial

- **Visión general de Claude Code**

<https://code.claude.com/docs/en/overview>

Herramientas integradas que usa el agente de terminal.

- **Visión general de tool use**

<https://docs.claude.com/en/docs/agents-and-tools/tool-use/overview>

Herramientas a medida frente a las integradas del SDK: el mismo apretón de manos.

- **MCP**

<https://docs.claude.com/en/docs/agents-and-tools/mcp>

Las herramientas de terceros se sientan junto a las integradas en la lista de permitidas.

- **Límites de tasa**

<https://docs.claude.com/en/api/rate-limits>

Los bucles de búsqueda web pueden ser caros.

Study notes distilled from the UST CCD Foundation masterclass. Prefer Anthropic's current docs over session anecdotes. Not official exam material.